

دانشکدهٔ ریاضی و آمار



برنامه سازی پیشرفته

مدرس: دکتر فاطمه منصوری

پروژه پایان‌ترم: ToTheTheath

محمد ملایی، داوود نصرتی امیرآبادی

ویرایش اول

نیم‌سال دوم ۱۴۰۲-۱۴۰۱

فهرست مطالب

۱ **مقدمه** ۴

۲ **فایل main.py** ۴

۱.۲ تابع change_scene ۴

۲.۲ تابع change_background ۴

۳.۲ تابع connect_to_server ۴

۴.۲ تابع game_stat_receiver ۴

۵.۲ تابع handle_server_msg ۵

۶.۲ تابع find_character ۵

۷.۲ تابع extract_characters ۵

۸.۲ تابع start_game ۵

۹.۲ تابع start_new_game ۵

۱۰.۲ تابع handle_scence ۵

۱۱.۲ تابع main ۶

۳ **فایل ui.py** ۶

۱.۳ کلاس Colors ۶

۲.۳ تابع window ۶

۳.۳ تابع change_light_theme ۶
۴.۳ تابع draw_text ۶
۵.۳ تابع draw_center_text ۶
۶.۳ تابع draw_center_text_list ۷
۷.۳ تابع draw_fill_text ۷

۴ تابع handle_scence **۷**
۱.۴ تابع disable_move ۷
۲.۴ تابع disable_move ۷
۳.۴ تابع draw_move_option ۷
۴.۴ تابع draw_bars ۷
۵.۴ تابع show_playing_screen ۸

۵ فایل choose_character_scene **۸**
۱.۵ تابع draw_character_option ۸
۲.۵ تابع show_stats ۸
۳.۵ تابع change_current_character_index ۸
۴.۵ تابع show_choosing_character_screen ۸

۶ فایل show_ending_scene **۸**
۱.۶ تابع show_ending_scene ۸

۷ فایل player **۹**
۱.۷ کلاس Player ۹

۸ فایل server.py **۹**
۱.۸ تابع process_move ۹
۲.۸ تابع start_match ۹
۳.۸ تابع handle_player_message ۹
۴.۸ تابع handle_player_disconnection ۹
۵.۸ تابع handle_player_connection ۹

۶.۸ تابع start_move ۹

۹ فایل Fight **۱۰**

۱.۹ کلاس Fight ۱۰

۲.۹ متد send_stat ۱۰

۳.۹ متد is_move_successful ۱۰

۴.۹ متد strike ۱۰

۵.۹ متد magic ۱۰

۶.۹ متد fight ۱۰

۱۰ فایل log.py_ **۱۰**

۱.۱۰ کلاس Log ۱۰

۲.۱۰ متد start_logging ۱۱

۳.۱۰ متد round_logging ۱۱

۴.۱۰ متد finish_logging ۱۱

۱ مقدمه

توضیحات ذیل، کلیاتی من باب عملکرد کلی بازی ToTheTheath است که به صورت جزئی به بررسی عملکرد کلاس ها، توابع و متد های برنامه می‌پردازد.

۲ فایل main.py

وظیفه اجرای بخش client بازی را به عهده داشته و با server ارتباط برقرار می‌کند. ابتدا یک سوکت با نام sc ایجاد شده، سپس طول و عرض صفحه اجرایی بازی تنظیم گردیده، با مشخص کردن رنگ پس زمینه، عنوان پنجره اجرایی، آدرس و پورت سوکت، FPS و صحنه ابتدایی (مقدار SCENE) به اجرای بازی می‌پردازد.

۱.۲ تابع change_scene

با دریافت یک scene از نوع str، متغیر SCENE را در حین اجرای بازی عوض می‌کند. این متغیر در ادامه توسط تابع handle_scences بررسی شده و موجب تعویض صحنه بازی خواهد شد. لازم به ذکر است که مقدار اولیه این متغیر connecting تنظیم شده است تا پنجره مربوطه را در ابتدای اجرای بازی نمایش دهد.

۲.۲ تابع change_background

این تابع با گرفتن رنگ background و با توجه به مقدار متغیر light_theme، تم بازی را (که شامل رنگ فونت ها و دیگر المان ها می‌شود) رنگ بندی کلی بازی را بسته به کاراکتر انتخابی توسط کاربر تغییر می‌دهد.

۳.۲ تابع connect_to_server

از سوکتی که در ابتدا ساختیم استفاده کرده و به عنوان اولین پیام دریافتی از سرور، لیستی از دیکشنری های حاوی اطلاعات کاراکتر ها را دریافت کرده و آنها را در متغیر characters قرار می‌دهد. در سری پیام های بعدی به کمک تابع game_stat_reveiver به بررسی حرکات بازیکنان خواهد پرداخت. در صورت دریافت ارور و عدم اتصال به سرور، ۱ ثانیه صبر کرده و سپس دوباره تلاش می‌کند.

۴.۲ تابع game_stat_receiver

منتظر پیام سرور بوده و بعد از decode کردن آن، به handle کردن پیام سرور توسط تابع handle_server_msg می‌پردازد.

۵.۲ تابع handle_server_msg

پیام های ارسالی توسط سرور به صورت دیکشنری هستند و با توجه به header شان، به دسته های زیر تقسیم می‌شوند:

- Namechosed: حریف کاراکتر خود را انتخاب کرده است، که در متغیر enemy ذخیره خواهد شد.
- Change_stat: میزان health یا magic کاربر توسط سرور ارسال شده و در سمت client تنظیم می‌شود.
- End: یکی از بازیکنان برنده بازی شده و بازی اتمام یافته است.

۶.۲ تابع find_character

با دریافت نام یک کاراکتر از مجموعه شخصیت های بازی، مقادیر و اطلاعات مربوط به آن را از دیکشنری کاراکتر ها برمی‌گرداند.

۷.۲ تابع extract_characters

پیام دریافتی از سرور که شامل اطلاعات کاراکترهاست به صورت str است و برای تبدیل آن به متغیری قابل پیمایش از کتابخانه json و تابع loads استفاده می‌شود.

۸.۲ تابع start_game

پس از انتخاب کاراکتر توسط کاربر، صفحه بازی به صحنه انتظار تغییر یافته و پیام انتخاب کاراکتر به سرور ارسال میشود.

۹.۲ تابع start_new_game

در صورت اتمام کامل بازی و برنده شدن یکی از بازیکنان و تمایل ایشان به شروع دوباره بازی، این تابع صدا زده شده و مقادیر کاراکتر ها به حالت پیشفرض برمی‌گردند تا دور جدید بازی آغاز شود.

۱۰.۲ تابع handle_scence

با دریافت event های پای گیم (که شامل تمام تحرکات کاربر و برنامه در طول اجرای بازی است) و بررسی متغیر ،SCENE به تغییر صحنه بازی می‌پردازد. به طور کلی بازی دارای ۶ صحنه اصلی است که عبارت‌اند از:

- choosing_character
- waiting
- connecting
- playing

- connection_error
- ending

۱۱.۲ تابع main

با تنظیم Clock کتابخانه Pygame به اجرای بازی با FPS ۶۰ پرداخته و به سرور متصل می‌شود.

۳ فایل ui.py

توابع مورد نیاز برای ایجاد رابط گرافیکی بازی را در برمی‌گیرد. در ابتدا آدرس فونت (های) مورد نیاز بازی تنظیم می‌شود.

۱.۳ کلاس Colors

حاوی سه مقدار WHITE BLACK، و GRAY است که در تنظیم Theme اصلی بازی نقش دارند. بازی در حالت اولیه در LIGHT_THEME قرار دارد. در ادامه نیز یک شئ از کلاس Colors ایجاد می‌شود.

۲.۳ تابع window

Pygame برای پردازش صفحه بازی نیاز به صدا زدن تابع window دارد، به همین دلیل برای جلوگیری از تکرار مکررات و سهولت در render هر فریم از بازی، از تابع window استفاده می‌شود.

۳.۳ تابع change_light_theme

رنگ های شئ از پیش تعریف شده‌ی colors را در صورت تغییر theme از light به dark تعویض می‌کند.

۴.۳ تابع draw_text

با دریافت متن، مختصات و رنگ مورد نظر، به نوشتن آن می‌پردازد.

۵.۳ تابع draw_center_text

با توجه به نیاز مکرر در طول اجرای بازی به نوشتن یک متن در مرکز صفحه، نوشتن یک تابع جداگانه برای محاسبه مختصات مرکز و نمایش متن در نقطه مورد نظر نیاز است.

۶.۳ تابع draw_center_text_list

در بخشی از بازی، متون مشخصی به صورت لیست چاپ خواهند شد که وظیفه انجام این کار به عهده این تابع است. بدین صورت که با محاسبه تعداد متون و ارتفاع کادر دربردارنده هر عضو از لیست، آن ها را در مرکز صفحه و زیر هم نمایش می‌دهد.

۷.۳ تابع draw_fill_text

یک مستطیلی توپر با مختصات دریافتی در صفحه رسم می‌کند.

۴ تابع handle_scence

صحنه بازی را تغییر می‌دهد. به طور کلی، بازی در صحنه playing_scene مشتمل بر دو صحنه است.

- showing_result
- choosing_move

۱.۴ تابع disable_move

در صورت اتمام magic_power بازیکن، دیگر امکان استفاده از آن در حین انجام بازی وجود نخواهد داشت. در نتیجه تا زمان پرشدن میزان لازم، انتخاب این حرکت غیرفعال خواهد شد.

۲.۴ تابع disable_move

حرکات غیرفعال شده را فعال می‌کند.

۳.۴ تابع draw_move_option

حرکات غیرفعال را خاکستری کرده و مابقی گزینه های موجود را در صفحه رسم می‌کند.

۴.۴ تابع draw_bars

با دریافت میزان health و magic هر بازیکن، به رسم آن در بالای صفحه می‌پردازد.

۵.۴ تابع show_playing_screen

صفحه مربوط به حرکات مجاز در بازی است و وظیفه بررسی و handle حرکات کاربر را دارد. در ادامه، در صورت انتخاب حرکت توسط کاربر، پنجره های مربوط به result showing move، enemy's for waiting نیز نمایش داده خواهند شد.

۵ فایل choose_character_scene

وظیفه نمایش صفحه اولیه بازی که شامل اطلاعات کاراکتر های مختلف است را به عهده دارد.

۱.۵ تابع draw_character_option

نام کاراکتر را ترسیم می‌کند.

۲.۵ تابع show_stats

مشخصات مربوط به هر کاراکتر را نمایش می‌دهد.

۳.۵ تابع change_current_character_index

با تغییر هر کاراکتر، index کاراکتر انتخابی تغییر کرده و به دنبال آن background عوض می‌شود.

۴.۵ تابع show_choosing_character_screen

در پنجره انتخاب کاراکتر، کاربر می‌تواند با Keys Arrow از روی کیبورد به تعویض کاراکترانتخابی بپردازد. این تابع وظیفه handle کردن دستورات کاربر را دارد و با بررسی تغییرات انجام شده، کاراکتر های مختلف را به کاربر نشان می‌دهد.

۶ فایل show_ending_scene

۱.۶ تابع show_ending_scene

صفحه نهایی بازی را نشان می‌دهد که در آن بازیکن برنده یا بازنده اعلام خواهد شد.

۷ فایل player

۱.۷ کلاس Player

صرفا جهت سهولت دسترسی به property های یک Player در طول بازی، این کلاس تعریف شده است. مشخصه های مربوطه به تفصیل در توضیحات فایل player در سمت Server توضیح داده خواهد شد.

۸ فایل server.py

۱.۸ تابع process_move

کاربری که اولین حرکت را بزند پیامی ارسال می‌کند که حرکت او و حریفش مورد تجزیه و تحلیل قرار می‌گیرند. بازیکن و حریف او تشخیص داده شده و حرکات آنها توسط یک شئ از کلاس Fight صورت می‌پذیرد. در نهایت حرکات آنها به حالت اولیه بازگشته و Round بعدی آغاز می‌شود.

۲.۸ تابع start_match

دو پلیر بازی بعد از انتخاب کاراکتر های خود به سرور اطلاع می‌دهند و سرور با استفاده از این تابع اعلان شروع بازی را ارسال کرده و به هر بازیکن نام حریف خود را ارسال می‌کند. در پایان نیز یک شئ از کلاس Log برای گرفتن لاگ در طول بازی ساخته می‌شود.

۳.۸ تابع handle_player_message

پیام های ارسالی از طرف client را بررسی می‌کند. پیام ها می‌توانند اعلان هایی برای شروع بازی یا حرکت انجام شده توسط کاربر باشند.

۴.۸ تابع handle_player_disconnection

اگر اتصال کاربر در هنگام اجرای برنامه قطع شود این تابع اتصالات را بررسی می‌کند. اگر کاربر همان waiting_player باشد، waiting_player خالی می‌شود. در صورتی که در میانه بازی باشد، به حریف پیغام قطع شدن حریف را می‌دهد.

۵.۸ تابع handle_player_connection

پیام های ارسالی توسط کاربر را دریافت می‌کند.

۶.۸ تابع start_move

Server را آماده دریافت پیام از سمت client کرده و به کمک thread ها، از کلاینت های مختلف پیام می‌گیرد.

۹ فایل Fight

۱.۹ کلاس Fight

با دریافت دو بازیکن از نوع ،Player نتیجه رقابتشان را در آن Round از بازی برمی‌گرداند. در ضمن یک شئ از کلاس Log نیز به آن داده می‌شود تا حرکات بازیکنان log شود.

۲.۹ متد send_stat

اطلاعات مربوط به هر بازیکن را به خود بازیکن ارسال می‌کند.

۳.۹ متد is_move_successful

با دریافت درصد موفقیت یک حرکت، یک مقدار bool برمی‌گرداند که نمایانگر موفقیت یا عدم موفقیت حرکت است.

۴.۹ متد strike

با دریافت بازیکن strike زننده به عنوان ورودی، در صورت موفقیت حرکت، از میزان health حریف کاسته شده و پیامی برای هر دو بازیکن ارسال می‌شود.

۵.۹ متد magic

با دریافت بازیکن magic کننده به عنوان ورودی، در صورت موفقیت حرکت، از میزان health حریف و magic خود بازیکن کاسته شده و پیامی برای هر دو بازیکن ارسال می‌شود.

۶.۹ متد fight

هسته مرکزی کلاس Fight است که حرکات بازیکنان را Handle میکند. در صورت پیروزی هر یک از بازیکنان، تابع آن بازیکن را خروجی داده و آن را به logger پاس می‌دهد. در پایان نیز نتیجه را به logger ارسال کرده تا در فایل ذخیره شود. اطلاعات توسط متد send_stat به بازیکنان ارسال شده و نتیجه fight نیز پاک می‌شود.

۱۰ فایل _log.py

۱.۱۰ کلاس Log

با دریافت دو بازیکن از نوع ،Player به ضبط اطلاعات مربوط به آنها در یک فایل در مسیر FILE_NAME می‌پردازد.

۲.۱۰ متد start_logging

زمان آغاز بازی و address هر بازیکن به همراه کاراکتر برگزیده وی را ذخیره می‌کند. سپس یک واحد به تعداد round های بازی که در آغاز ۰ بود می‌افزاید.

۳.۱۰ متد round_logging

یا دریافت نتیجه حرکت هر بازیکن، نوع حرکت، نتیجه آن، میزان health و magic در پایان آن راند را می‌نویسد.

۴.۱۰ متد finish_logging

با دریافت بازیکن برنده، نتیجه بازی را ذخیره می‌کند.

پایان.